



Automatic GPU Memory Management for Large Neural Models in TensorFlow

Tung D. Le
IBM Research - Tokyo
Tokyo, Japan
tung@jp.ibm.com

Yasushi Negishi
IBM Research - Tokyo
Tokyo, Japan
negishi@jp.ibm.com

Haruki Imai
IBM Research - Tokyo
Tokyo, Japan
imaihal@jp.ibm.com

Kiyokuni Kawachiya
IBM Research - Tokyo
Tokyo, Japan
kawatiya@jp.ibm.com

Abstract

Deep learning models are becoming larger and will not fit in the limited memory of accelerators such as GPUs for training. Though many methods have been proposed to solve this problem, they are rather ad-hoc in nature and difficult to extend and integrate with other techniques. In this paper, we tackle the problem in a formal way to provide a strong foundation for supporting large models. We propose a method of formally rewriting the computational graph of a model where swap-out and swap-in operations are inserted to temporarily store intermediate results on CPU memory. By introducing a categorized topological ordering for simulating graph execution, the memory consumption of a model can be easily analyzed by using operation distances in the ordering. As a result, the problem of fitting a large model into a memory-limited accelerator is reduced to the problem of reducing operation distances in a categorized topological ordering. We then show how to formally derive swap-out and swap-in operations from an existing graph and present rules to optimize the graph. Finally, we propose a simulation-based auto-tuning to automatically find suitable graph-rewriting parameters for the best performance. We developed a module in TensorFlow, called *LMS*, by which we successfully trained ResNet-50 with a 4.9x larger mini-batch size and 3D U-Net with a 5.6x larger image resolution.

CCS Concepts • Software and its engineering → Memory management.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ISMM '19, June 23, 2019, Phoenix, AZ, USA

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6722-6/19/06...\$15.00

<https://doi.org/10.1145/3315573.3329984>

Keywords large neural models, computational graphs, simulator, GPU memory

ACM Reference Format:

Tung D. Le, Haruki Imai, Yasushi Negishi, and Kiyokuni Kawachiya. 2019. Automatic GPU Memory Management for Large Neural Models in TensorFlow. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on Memory Management (ISMM '19)*, June 23, 2019, Phoenix, AZ, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3315573.3329984>

1 Introduction

Deep learning models are effective for solving complex signal-processing problems such as those in computer vision, speech recognition, and natural language processing. However, training a model is time-consuming, often taking days to weeks. The training is based mainly on matrix multiplications; therefore, it is often accelerated by using accelerators such as GPUs. In 2012, GPUs were used for training a deep learning model for the first time. The model was a deep convolutional neural network of 16 layers, called *AlexNet* [12], and it achieved outstanding image classification results in the ILSVRC-2012 competition¹ with a top-5 test error rate of 15.3%. Since then, GPUs have been popular for deep learning.

After the success of AlexNet, deep learning has evolved quickly for a broader spectrum of applications. Models are deeper (including more layers) and larger, e.g., ResNet-1001 consists of 1001 layers and is much deeper than AlexNet [9]. Thus, models are sometimes too large to be fit for training due to the memory limitations of GPUs (the latest GPUs have 16 or 32 GB of memory).

From a hardware viewpoint, GPUs should be designed to have a larger physical memory, but increasing physical memory is expensive since it is on-chip high bandwidth memory. From a software viewpoint, there are three main approaches to solving this problem. The first one is reducing memory consumption by reusing memory regions for different computations [16], compressing a neural network [5], or using low precision [7]. The second is re-computing some

¹<http://www.image-net.org/challenges/LSVRC/2012/>

of the computations from checkpoints [17]. The third is using external memory such as CPU memory to temporarily store intermediate results during training [13, 14]. Despite the many approaches that have been proposed, there is a lack of fundamental theories that guide the management of memory consumption for large neural models. This leads to the consequence that it is non-trivial to apply the existing approaches to different deep learning frameworks and to combine them together.

We pursued the third approach of using external memory because it often helps with training a larger model compared with the other approaches, and it can be theoretically applied to any model. Different from the previous studies involving swapping data from GPU memory to external memory, and vice versa, *in an ad-hoc manner*, we propose an approach based on formal rules for graph rewriting. The advantage of a graph-rewriting-based approach is that we can formally derive new operations that are added to a graph correctly. This is important, in particular, for editing a graph since an ad-hoc approach is error-prone and might accidentally introduce invalid execution. Furthermore, this helps automatically tune graph-rewriting parameters to achieve the best performance in a systematic way as we will show in this paper. As a result, we achieve a fundamental method that is able to automatically manage GPU memory for a given large neural model by statically analyzing a computational graph of it. Our contributions in this paper are as follows.

- We formalize graph execution in TensorFlow [1], a popular deep learning framework, by using a categorized topological ordering, helping us analyze the memory consumption of a model. We then reduce the problem of fitting a large model into a memory-limited accelerator to the problem of reducing operation distances in the categorized topological ordering.
- We formally derive swap-out and swap-in operations from an existing graph, those used to exchange intermediate results between GPUs and CPUs. Derivation is based on some rules in program transformation with correctness guarantee, which helps us understand the nature of swapping operations. A fusion rule is applied to optimizing the amount of data transferring between GPUs and CPUs. As a result, we introduce a set of graph-rewriting parameters that control how a graph is rewritten.
- We propose a simulation-based auto-tuning for automatically finding suitable graph-rewriting parameters so that a large neural model can fit and run fast on GPUs. A simulator is proposed to simulate how high memory consumption is, given a graph and graph-rewriting parameters. The simulator is able to simulate swap-out and swap-in operations without actually adding them into a graph.

- We develop a module in TensorFlow, called *Large Model Support* (LMS). With LMS, we were able to train ResNet-50 [8], a popular model for image recognition, with a 4.9 times larger mini-batch size and 3D U-Net [6, 11], a popular model for 3D image segmentation, with a 5.6 times larger image resolution.

The rest of the paper is organized as follows. In Section 2, we discuss related work. In Section 3, we revise the concept of the computational graph and present the semantics of the graph. In Section 4, we define a categorized topological ordering and use it to derive swap-out and swap-in operations and propose some improvements. We also use it to explain why training a model consumes much memory. In Section 5, we introduce a simulator to simulate the GPU memory consumption of a computational graph. Then, we propose an auto-tuning algorithm that uses the simulator to search for graph-rewriting parameters. In Section 6, we discuss our LMS module that implements our approach in TensorFlow. In Section 7, we present the experimental results. Section 8 summarizes the key points and is a discussion of future work.

2 Related Work

The most intuitive method for training large models is using Unified Memory [15], a single memory address space accessible from both CPUs and GPUs. Enabling Unified Memory is simple, but its performance is very poor compared with custom methods that manually offload and prefetch data. Shirahata et al. [16] proposed a reduction approach in which the memory regions that are allocated for the forward phase can be reused during the backward phase. Rhu et al. [14] proposed a different approach of managing runtime memory by virtualizing the memory usage of neural networks against both GPU and CPU memory. During training, only the current layer is active and consumes GPU memory, while the other layers' data are swapped out to the CPU memory. Such a swapping method is similar to the eviction mechanism in cache management but specific to neural network training algorithms. This approach performed better than using Unified Memory. Meng et al. [13] took the same approach as [14] for TensorFlow by swapping tensors from GPU memory to CPU memory and vice versa. Both approaches in [14] and [13] looked for tensors (edges) between the forward and backward phases, and swapped the tensors between memories. This is different from our approach where we take care of the whole graph, e.g., looking for links in the whole graph, and the forward and backward phases do not need to be determined explicitly. Furthermore, Meng et al. did not discuss how to derive swap-out and swap-in operations. Our work was inspired by Meng et al.'s idea but formally defines transformation rules for graph rewriting so that the correctness of the transformed computational graph is guaranteed. Apart from using CPU memory as temporary

memory for computation, Tianqi Chen [17] proposed an approach of gradient-checkpointing, in which checkpoint vertices in a computational graph are automatically defined by using graph partitions. Parts of the graph in between checkpoint vertices are re-computed during the backward phase. The forward phase is thus generally computed twice. Wang et al. [18] combined both swapping and re-computation into a single framework.

3 Computational Graphs

The computational graph is a core concept in deep learning. In TensorFlow, a neural network defined by users is internally represented by a computational graph of operations. Operations in the graph are then placed on devices such as CPUs and GPUs for execution. In this section, we revise the concept of the computational graph in TensorFlow [1] to make its semantics more consistent.

3.1 Definition

Definition 1. (Computational graph) Let $G = (V, E, \omega, \tau)$ be a vertex and edge-labeled directed graph, where V is the set of vertices in G , $E \subseteq V \times V$ is the set of edges in G , $\omega : V \rightarrow (O, \text{Bool})$ is a function that maps each vertex to a tuple of an operation $o \in O$ and a Boolean value that indicates whether the operation is parameterized or not, and $\tau : E \rightarrow (T, \text{ACT})$ is a function that maps each edge to a tuple of a value of data type T and an action in ACT where $\text{ACT} = \{\text{"data"}, \text{"control"}\}$.

A computational graph is a way of expressing mathematical expressions in which each vertex is an operation with inputs of incoming edges and outputs of outgoing edges. In deep learning, a computational graph is used to express computation for a neural network in which operations are represented by vertices, and data are represented by edges.

Each vertex in a computational graph is an operation whose inputs are data on incoming edges and outputs are data on outgoing edges. Data are often *tensors*—multi-dimensional arrays. Since tensors are also used to store the internal states of a neural network, e.g., learning weights and biases of hidden layers in a neural network, which are regularly updated, we classify operations into *normal operations* and *parameterized operations*. Normal operations are normal functions that define a mapping from inputs to outputs. Parameterized operations have internal states that can be updated. A *variable* is a special parameterized operation that updates its internal variable by using an identity operation². A *constant* is a special case of a variable whose value is set once and is never updated.

Each edge in a computational graph is a tuple of a tensor and a value that indicates edge type. There are two types of edge: “data”, and “control”. “Data” edges contain actual data or tensors produced by an operation, while “control” edges do not contain data and only indicate an execution

ordering between two operations. Consider edge (u, v) from operation u to operation v . If $\tau(u, v) = (t, \text{"data"})$, tensor t is produced by u and consumed by v . If $\tau(u, v) = (t, \text{"control"})$, tensor t is a “dump” tensor, having no data. The “control” edge (u, v) says that v must execute after u , and u is called a *control dependency operation*.

3.2 Notations and Semantics

Table 1 lists notations that represent different vertices and edges in a graph. Function composition is denoted as “ $\circ$ ”, and, from its definition, we have $(f_2 \circ f_1)x = f_2(f_1(x))$. When discussing about execution ordering, we sometimes use “ $f_1 \rightarrow f_2$ ” to represent “ $f_2 \circ f_1$ ”. A list is denoted as “[]”, and a set is denoted as “{ }”. A vertex represents an operation that accepts a list of inputs and returns a single output.

An operation in a computational graph is generally triggered to execute when all of its incoming edges have data. The operation generates data on its outgoing edges; then, other operations are repeatedly triggered in the same manner. This procedure ends when all of the reachable operations are executed and all of the reachable edges are filled with data. In other words, each of the reachable operations, except for variables, is executed once. It is worth noting that there is no way to trigger the execution of a graph. At the beginning of computation, there is no way to set a value for a tensor. To solve this problem, TensorFlow uses a *placeholder*. A placeholder keeps a tensor and users feed data to the placeholder to trigger its consuming operations.

Triggering an operation depends not only on data availability on incoming edges but also on control dependency edges. “Control” edges do not have data and are used to manually control the execution order of an operation. If (u, v) is a “control” edge, v is postponed until u finishes even though the other in-coming edges of v have already had data.

Variables in a computational graph are used to store learnable parameters, input, and output. Variables are special and make a computational graph for deep learning different from a general dependency graph. Because a variable has an internal state, defining its semantics is non-trivial in the context of a graph. At the beginning, all variables must be initialized with values fed by users or random values generated by a distribution. During training, the variables are updated by a learning optimizer. The result is that a variable can be visited more than once, and this may introduce cycles if the semantics is ambiguous. In TensorFlow [1], to avoid cycles, variables have only out-going edges. As a result, write operations to variables are also out-going edges and they update the input variable, leading to confusing semantics that does not follow the triggering process of operations.

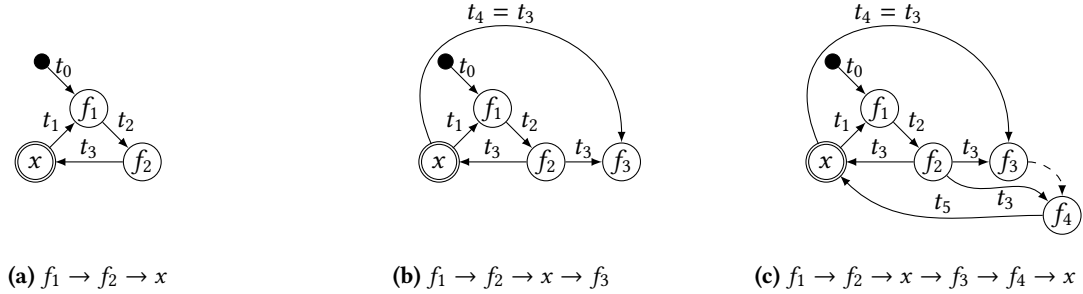
The remainder of this section introduces a consistent semantics for variables, where we propose two following conventions for variables:

- C1. A variable cannot trigger its out-going operations.

²The identity function accepts a value and returns the same value

Table 1. Notations for computational graph

	Notation	Definition	Meaning
Vertices		$\omega(f) = (f, False)$ $t_4 = f([t_1, t_2, t_3])$	Normal operation f , taking list of inputs t_1, t_2, t_3 and returning output t_4 .
		$\omega(x) = (x, True)$ $t = x$	Parameterized operation or variable x .
Edges		$\tau(f, g) = (t, "data")$	f produces data t , and g reads t as input. This edge represents $g \circ f$.
		$\tau(f, x) = (t, "data")$ $x = t$	f produces data t that is used to update variable x . t and x must have same type.
		$\tau(f, g) = (_, "control")$	g consumes no data from f , and g does not execute unless f finishes.
		placeholder	Placeholder keeps tensor t . When t is fed data, g is triggered to execute.
Devices		f^G (GPU operation)	Operation f is executed on GPU.
		f^C (CPU operation)	Operation f is executed on CPU.

**Figure 1.** Examples of semantics of variables.

C2. Among operations that consume the same tensor, variables have the highest execution priority.

Convention 1 comes from the fact that a variable is an operation that updates its internal state. Hence, it should have no ability to trigger other operations. As a result, this convention is for ensuring that graph execution does not go into an infinite loop and cycles can be unfolded. Let us consider the example graph in Fig. 1a. The graph has the following execution ordering: “ $f_1 \rightarrow f_2 \rightarrow x$ ”. First, variable x is initialized by users. Then, f_1 is triggered by feeding data to tensor t_0 . Then, f_1 is executed and triggers operation f_2 . Finally, x is updated with the output of f_2 , and the computation finishes because variable x cannot trigger f_1 again due to Convention 1.

Convention 2 defines an execution priority for variables. In Fig. 1b, the execution ordering should be “ $f_1 \rightarrow f_2 \rightarrow x \rightarrow f_3$ ”. In this graph, both x and f_3 consume tensor t_3 . According

to Convention 2, x will be executed before f_3 . Hence, the input t_4 of f_3 will use the latest value of x , that is, t_3 .

In Fig. 1c, the graph has a new operation, f_4 , that consumes the output of f_2 , executes computation, and updates x . Without the control edge from f_3 to f_4 , after f_2 is executed, f_3 and f_4 can be executed in parallel because they do not depend on each other. Since they both access x , i.e., f_3 reads x , and f_4 writes to x , a control edge is necessary to ensure that they access x in order. If we remove the control edge from f_3 to f_4 and replace it with a control edge from f_4 to f_3 , then x will be updated by using the output tensor t_5 of f_4 first according to Convention 2. In this case, the input t_4 of f_3 has a value of t_5 instead of t_3 .

3.3 Training using Back-propagation

Training a neural network involves minimizing an objective function, f , that measures the distance between a ground truth value and predicted value. The objective function is a

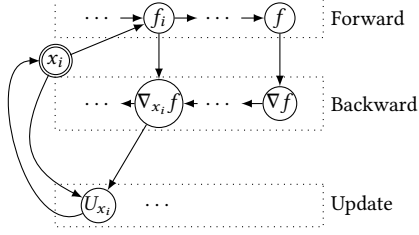


Figure 2. How variable is used and updated in training. Dotted-line boxes are used to separate operations in different phases.

composition of multiple functions that accept learnable parameters, and the gradient descent algorithm is often used to minimize the function. Optimization is an iterative procedure that updates learnable parameters so that the objective function is minimized, in which each training iteration consists of three phases: a *forward phase* to compute the objective function, a *backward phase* to compute gradients of the objective function with respect to learnable parameters, and a *update phase* to update learnable parameters with their gradients. The backward phase is done via back-propagation for efficiency, starting from the objective function and propagating back gradients through the functions. At the beginning of an iteration, the memory areas for tensors, except learning parameters, are released. Input tensors are then fed with new data and trigger the iteration. Because a training dataset is often very big, each iteration takes only a subset (*mini-batch*) of examples extracted from the training dataset as its input tensor. The number of examples in a mini-batch (or *mini-batch size*) will affect the size of the input tensor and also other tensors in a computational graph. In general, increasing the mini-batch size will make a model larger.

Figure 2 shows how learnable parameters (represented by variables) are updated during training. In the forward phase, variable x_i is an input to function f_i , outputs from f_i are used in the later function, and, finally, a loss value is produced by objective function f . In the backward phase, we compute gradients of f with respect to learnable parameters. Function $\nabla_{x_i} f$ computes the gradient of f with respect to x_i , which requires f_i 's output as one of its inputs. Finally, x_i is updated by a function U_{x_i} during the update phase.

3.4 Device Placement

In TensorFlow, each operation in a computational graph is placed on a device such as a GPU or CPU. Communication between two devices automatically occurs if an operation on one device consumes a tensor produced by another operation on the other device. In fact, TensorFlow adds a pair of two operations, “send” and “receive”, to a graph for exchanging a tensor. In this paper, we do not show these communication operations when drawing graphs.

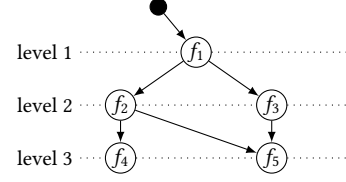


Figure 3. Categorized topological ordering $[\{f_1\}, \{f_2, f_3\}, \{f_4, f_5\}]$.

3.5 Garbage Collection

If a tensor is no longer used in TensorFlow, it is released by TensorFlow garbage collection. Every tensor is assigned a reference count that is initialized with the number of operations that consumes the tensor. Each time a consuming operation is triggered to execute, the reference count is decreased by one. If the reference count reaches zero, the tensor is available to be released. In other words, the lifetime of a tensor is from the operation generating it to the last operation consuming it.

4 Graph Rewriting

4.1 Categorized Topological Ordering

According to the way operations are triggered, we found that the process of triggering operations can be represented by a *categorized topological ordering*. We define it in this section and use it in the remainder of this paper for graph rewriting.

To say vertex v depends on vertex u means that there exists a directed edge from u to v . A categorized topological ordering of a graph is the topological ordering in which vertices are categorized into sets. We define a categorized topological ordering to be a linear ordering of sets of vertices such that vertices in set x have no dependencies *a)* in that set and *b)* in any set y that comes after x in the order. We represent a categorized topological ordering as a list of sets with one-based indexing. To say a vertex has *level i* means that the set containing the vertex is the i -th element in the list. We define a function, γ , to return the level of a vertex.

Figure 3 shows an example of categorized topological ordering. Operation f_1 is triggered first, and f_2 and f_3 are then triggered, and, finally, f_4 and f_5 are triggered. Operations at the same level, say f_2 and f_3 , have no dependencies on each other, and they can be executed in a parallel or sequential way depending on whether they are placed on the same device or different devices. On the same device, operations at a level cannot be triggered until all of the operations at the previous level finish.

Before building a categorized topological ordering, it is necessary to pre-process a computational graph. A topological ordering is possible if and only if the graph has no directed cycles. However, our computational graph is cyclic because of the variables, though the semantics of variables guarantees that there is no infinite loop in execution. Recall

Table 2. Summary of parameters for graph rewriting.

Graph-rewriting parameters	Presented in	Meaning
<i>swapout_threshold</i>	Section 4.3	Edges with distance greater than <i>swapout_threshold</i> will be rewritten.
<i>swapin_ahed</i>	Section 4.5	How in advance a swap-in operation is triggered.
<i>swapin_groupby</i>	Section 4.4	If operations consuming the same tensor from multiple swap-in operations are within <i>swapin_groupby</i> distance, then the swap-in operations are fused into one swap-in operation.

that variables can not trigger other operations according to Convention 1. Hence, it is safe to remove variables from the graph because they do not contribute to the process of triggering operations.

To build a categorized topological ordering, we use a recursive algorithm. We build a set of all operations that have no dependencies and add the set to the ordering. These operations are then removed from the graph. Then, we repeat building another set. The above procedure finishes when all operations are removed from the graph.

The concept of the lifetime of a tensor can be easily represented by using a categorized topological ordering. Given edge (u, v) , the *distance* between u and v is defined to be $d(u, v) = \gamma(v) - \gamma(u)$. Let ts be a tensor produced by an operation, f , and $g_1, g_2, \dots, g_k$ be k operations that consume ts ; the lifetime of ts is computed as $\max\{d(f, g_1), d(f, g_2), \dots, d(f, g_k)\}$.

For brevity, from now on we omit the word “categorized” when referring to a categorized topological ordering.

4.2 Why Does a Model not Fit GPU Memory?

A computational graph or a neural network model is said to be too large to be trained with the memory limitations of GPUs if there is not enough memory to train the model. During training, there are many tensors that are kept in the GPU memory at a time. If the total memory for these tensors exceeds the GPU memory, an out-of-memory error happens, and the training stops. This is essentially because there are many tensors with a long lifetime in a computational graph. From the viewpoint of topological ordering, given edge (u, v) , the larger $d(u, v)$ is, the longer its tensor is kept in GPU memory.

In the remainder of the paper, we will present how to rewrite a large graph so that training it is possible with limited GPU memory. In general, our idea is temporally sending long lifetime tensors from a GPU to a CPU and sending them back to the GPU when necessary. In other words, GPU memory acts like a cache while data are actually stored on host memory.

4.3 Deriving Swapping Operations

As presented in Section 4.2, edges with a large distance cause more tensors to be kept in the GPU. The purpose of rewriting is to reduce the distance of edges. In the context of statically rewriting a graph, we introduce a user-defined parameter,

called *swapout_threshold* (Table 2). Edges with a distance greater than *swapout_threshold* are rewritten.

Figure 4 shows an example of our rewriting procedure.

To put a tensor residing in GPU memory on CPU memory, we derive operations to automatically send the tensor to the CPU and send it back to the GPU. Let us consider an edge, (f_1, f_2) , where $\tau(f_1, f_2) = (t, _)$ and f_1, f_2 are GPU operations. Computation for this edge is as follows.

$$f_2^G \circ f_1^G \quad (1)$$

By definition of the identity operation, (1) can be rewritten as:

$$f_2^G \circ \text{id}^C \circ f_1^G \quad (2)$$

where id is an identity function, that is, $\text{id}(x) = x$.

Since id is a CPU operation, the output tensor of f_1 will be swapped out to the CPU memory for id immediately after f_1 finishes, and GPU memory is released. Similarly, the output tensor of id will be swapped in to the GPU for f_2 immediately after id finishes. We call function id in Equation 2 a *swap-out operation*.

However, Equation 2 does not help reduce memory consumption. Assume that f_1 is at level 1, i.e., $\gamma(f_1) = 1$, and f_2 is at level 100, i.e., $\gamma(f_2) = 100$, in the topological ordering. Since $d(f_1, f_2) = 99$, the lifetime of the tensor t in the GPU is 99. Our purpose for rewriting is to reduce the lifetime of t . When adding the swap-out operation id , id is at level 2, and we have $d(f_1, \text{id}) = 1$. As a result, the lifetime of t is reduced from 99 to 1. However, since $d(\text{id}, f_2) = 98$, the lifetime of the output tensor of id is 98. The result is that the output of id is swapped in too early and it is still kept in the GPU for a long time.

To solve the above problem, we need an additional operation for swapping in, and this operation can be derived from the swap-out operation.

An identity function can be rewritten as a composition of a function and its inverse function, that is, as follows.

$$\text{id} = f^{-1} \circ f \quad (3)$$

Substituting (3) into (2), we have the following.

$$f_2^G \circ (f^{-1})^C \circ f^C \circ f_1^G \quad (4)$$

Since id also has an inverse function, i.e., id , we choose id for f as follows (if one would like to reduce the memory consumption on the CPU, a pair of encoding and decoding

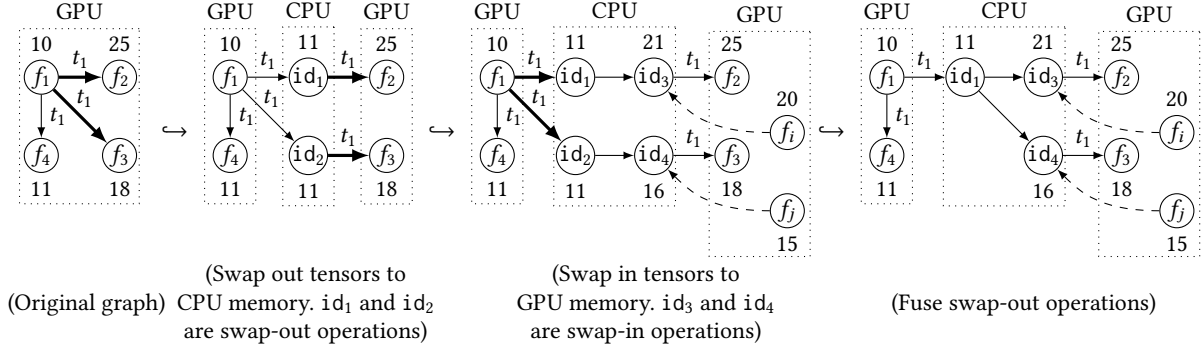


Figure 4. Example of graph rewriting for supporting large models. Thick edges in the leftmost subgraph are rewritten stepwise to produce the rightmost subgraph. Integers above or below vertex are level of that vertex in topological ordering. In this example, $swapout_threshold=5$, so edges from f_1 to f_2 and f_3 are rewritten. f_i and f_j are control dependency operations that trigger executions of swap-in operations id_3 and id_4 , respectively.

functions can be used for f instead of id).

$$f_2^G \circ id_2^C \circ id_1^C \circ f_1^G \quad (5)$$

In Equation 5, id_2 is used to swap a tensor into a device, and we call function id_2 a *swap-in operation*.

Again, Equation 5 is not enough because id_2 , being similar to id_1 , only reduces the lifetime of the output tensor of id_1 , and the lifetime of the output tensor of id_2 is still big. However, the important point is that we can manually trigger id_2 by using a control edges.

Recall that we assumed that $\gamma(f_1) = 1$ and $\gamma(f_2) = 100$. From Equation 5, we have $\gamma(id_1) = 2$. If there is no control edge to id_2 , then $\gamma(id_2) = 3$ and $d(id_2, f_2) = 97$. If there is a control edge from an operation g , $\gamma(g) = 97$, then $\gamma(id_2) = 98$ and $d(id_2, f_2) = 2$. In summary, together with a pair of swap-out and swap-in operations, control edges are important for controlling the lifetime of a tensor in a GPU. The insertion of control edges will be presented in detail in Section 4.5.

4.4 Fusing Swapping Operations

Equation 5 is not optimized because the output tensor of f_1 may be swapped out and swapped in multiple times since there may be multiple operations apart from f_2 consuming that tensor. In this section, we present two rules for improving Equation 5. Figure 4 shows computational graphs obtained by each of the rules.

4.4.1 Fusing Swap-out Operations

A tensor produced by an operation is often used by multiple operations, and it is redundant if the tensor is swapped out to CPU memory multiple times. Hence, it is recommended that swap-out operations of the same tensor always be fused into a single swap-out operation.

4.4.2 Fusing Swap-in Operations

Consider a situation in which multiple swap-in operations swap a tensor multiple times for multiple consuming operations. If the tensor is large and the consuming operations are close to each other, then swapping the tensor multiple times would introduce more overhead. In this case, it is better to fuse the swap-in operations into one swap-in operation. The tensor is swapped in only once and resides in GPU memory to be reused by the consuming operations. For example, in the right-most graph in Fig. 4, if f_2 and f_3 are close and t_1 is large, then we fuse id_3 and id_4 into a single swap-in operation. To determine how close two operations are, we introduce a user-defined parameter, called *swapi_groupby* (Table 2). Two operations are said to be close if the distance between them is less than or equal to *swapi_groupby*.

4.5 Inserting Control Edges

Control edges for swap-in operations are added to a computational graph to control when swap-in operations are triggered. Let us assume that an edge, (f_1, f_2) , is rewritten by using a swap-out operation, so , and swap-in operation, si . Replacing identity operations in Equation 5 with so and si , we have the following.

$$f_2^G \circ si^C \circ so^C \circ f_1^G \quad (6)$$

A control operation for the swap-in operation si must be chosen from a set of operations, V_c , where $\forall v \in V_c, \gamma(so) < \gamma(v) < \gamma(f_2)$ to guarantee the correctness of the computational graph. Let $k = \gamma(f_2) - \gamma(v)$ be the distance between f_2 and v . If k is too small, a tensor is swapped in too late, and f_2 has to wait for the tensor. If k is too large, a tensor is swapped in too early, and the tensor is kept in the device for a long time before being actually used by f_2 . A large value of k might cause training to consume more memory. Hence, control edges affect not only training performance but also memory consumption.

An ideal solution for choosing a control operation is having a cost model for computational graphs and using the model to prioritize operations. However, in the context of statically modifying a computational graph, we introduce a user-defined parameter, *swapin_ahead*, to handle the choosing of control operations. A control operation, g , for s_i is simply chosen from a set with the level, i.e., $\gamma(f_2) - \text{swapin_ahead}$, in the topological ordering. Furthermore, g must be lie on some of the paths from f_1 to f_2 to guarantee that g is triggered once f_1 is triggered.

5 Automatic Tuning of Graph-rewriting Parameters

There are three parameters that control graph rewriting: *swapout_threshold*, *swapin_ahead* and *swapin_groupby* (Table 2). To obtain the best performance, it is necessary to find a good combination of these parameters. In this section, we propose a novel approach to automatically tuning these parameters. Our idea is proposing a simulator that simulates how memory is allocated and deallocated given a graph. Then, an auto-tuning module, while exploring the search space, uses the simulator to check memory consumption for a given set of parameters.

5.1 Search Space of Graph-rewriting Parameters

Given a graph with $|V|$ vertices and $|E|$ edges, its topological ordering has $|L|$ levels. Each parameter's value ranges from 1 to $|L|$. In particular, parameter *swapout_threshold* is applied to the whole graph, while parameters *swapin_ahead* and *swapin_groupby* are per vertex. Hence, the search space for the three parameters is $((|L|) \times (|V| \times (|L|))^2)$. In the worst case, where $|L| = |V|$, the search space is $O(|V|^5)$. In this paper, to reduce the search space, we consider only that the three parameters are per the whole graph, i.e., *swapin_ahead* and *swapin_groupby* are shared by all vertices. In this case, the search space becomes $O(|V|^3)$.

5.2 Simulator

A simulator is used to simulate how high memory consumption is, given a computational graph. We assume that all operations in the graph are GPU operations, and the simulator simulates GPU memory consumption only.

There are two components in our simulator. The first one is for simulating memory consumption for all operations that exist in a graph. The second one is for simulating memory consumption for a specific set of graph-rewriting parameter values *without actually adding swap-out and swap-in operations into the graph*. For the second one, we can add swap-out and swap-in operations and then use the first one for simulation. However, this approach would introduce many redundant operations in the graph, and we would need to remove them from the graph after each simulation. TensorFlow does not provide an official way for deleting operations.

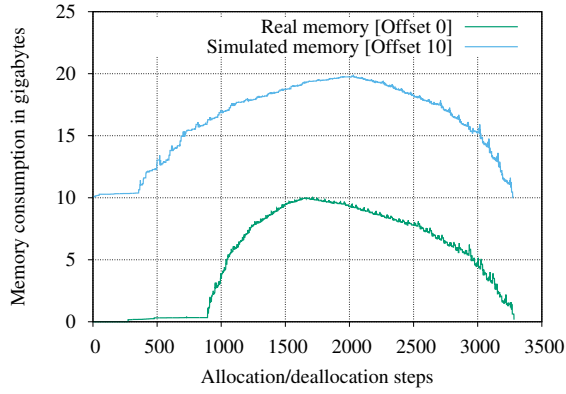
Users need to serialize a graph into a portable format such as a protocol buffer and mutate and load it back to TensorFlow. Another way is to make an operation dangle by re-routing its input and output edges. Both ways are expensive in graph modification and unnecessarily complex.

5.2.1 Simulating Existing Operations

To simulate memory consumption for all existing operations in a graph, we, step by step, trigger operations by using the topological ordering, starting with operations at level 1 and then going to higher levels up to the last one. Whenever an operation is triggered to run, we allocate and deallocate memory for its input and output tensors as follows. For each output tensor, we allocate for it an amount of memory that equals the tensor size, and we keep a reference count for the tensor that equals to the number of consuming operations. For each input tensor, if the tensor is not in the memory (for example, a tensor for input data like images and text), we allocate memory for it in the same way as we do for output tensors. If the tensor exists in the memory, its reference count is decreased by one. A garbage collection is called after each operation is triggered in order to release memory. The garbage collection checks the reference counts of all tensors in the memory, and it deallocates the memory of a tensor if the tensor's reference count reaches zero.

5.2.2 Simulating Swap-out and Swap-in Operations

Swap-out and swap-in operations are for introducing data transfer between the GPU and CPU. Hence, we need to know when a swap-out operation finishes and when a swap-in operation is triggered so that we can simulate memory deallocation and allocation correctly. Edges whose distances are greater than *swapout_threshold* are rewritten. To simulate edge rewriting without actually adding new operations into a graph, we update the reference counts of the tensors on the rewritten edges and mimic swap-out and swap-in operations as follows. If an edge is rewritten, we need to swap out its tensor and decrease its reference count once the swapping finishes. Since we do not know when a tensor is totally swapped out to the CPU, we immediately decrease its reference count. This means that a tensor on the edge is swapped out, and its memory is released immediately. Though swapping out a tensor does take time in a real operation, it does not affect the peak memory consumption we are simulating. This is because, in back-propagation training, the backward phase likely consumes more memory than the forward phase. A tensor swapped out needs to be swapped in at a good timing. Swapping in the tensor can be correctly simulated by using a control operation. By using *swapin_ahead*, we know which is the control operation for the tensor. Hence, after the control operation is triggered (actually, after the garbage collection is called for this operation), we allocate memory for all swap-in tensors that are triggered by the control operation. For each swap-in tensor, its reference count equals the number



(a) ResNet-50, mini-batch size of 128.

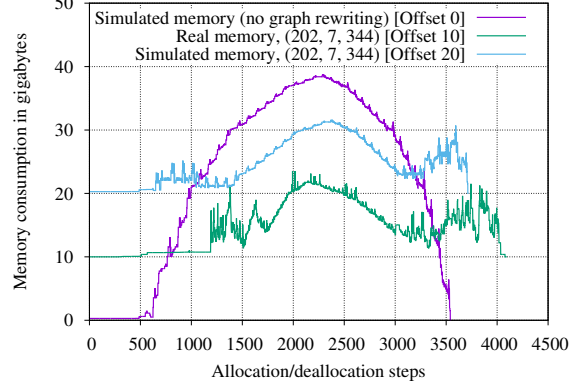
(b) ResNet-50, mini-batch size of 512, (202, 7, 344) are values for (*swapout_threshold*, *swapin_ahead*, *swapin_groupby*).

Figure 5. Comparison of real and simulated memory consumption. With mini-batch size of 128, the model fits in 16 GB of GPU memory, and graph rewriting is not applied. With mini-batch size of 512, the model does not fit in the GPU memory, and graph rewriting is applied. To visualize clearly, we shifted plots along the y-axis with an offset that is shown in legends.

of consuming operations, which depend on *swapin_groupby*. After that, the swap-in tensor is automatically deallocated by the last consuming operations.

5.2.3 Testing the Simulator

Figure 5 shows a comparison between the real and simulated memory consumptions for the neural network ResNet-50. Real memory consumption was obtained from TensorFlow by enabling its debug logs.

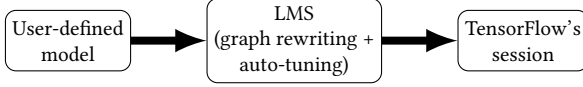
With a mini-batch size of 128 (Fig. 5a), the model fit in 16 GB of GPU memory, and graph rewriting was not applied. In this case, our simulator simulated only existing operations and did well. Though the simulated peak memory was a bit lower than the real one, the shapes of memory consumption were quite similar. These shapes are typical for back-propagation training when all data are in memory, in which the memory reaches its peak at the end of the forward phase and decreases during the backward phase.

With a mini-batch size of 512 (Fig. 5b), the model did not fit in 16 GB of GPU memory, and our simulator for all existing operations in this model also said that the peak memory consumption was about 40 GB (the plot with an offset of 0 in Fig. 5b). Hence, graph rewriting was applied, in which we used values (202, 7, 344) for (*swapout_threshold*, *swapin_ahead*, *swapin_groupby*). We will compare three parts in the plots at offsets of 10 and 20, that is, the left part, from steps 0 to 1700, showing the effectiveness of swap-out operations during the forward phase, the middle part, from steps 1700 to 3000, showing the memory residing in GPU memory, and the right part, from steps 3000 to 4500, showing the effectiveness of swap-in operations during the backward phase. For the left part, the simulator underestimated memory consumption while swapping out tensors.

For the middle part, the simulated memory consumption was quite similar to the real one, except for two spikes in the middle of their shapes. We found that these spikes came from the internal memory consumption of an operation, i.e., Conv2DBackpropInput. Though this operation has small input and output tensors (consuming about 260 megabytes), TensorFlow’s debug logs showed that the operation itself allocated and deallocated a block of 1.3 gigabytes during its computation. In TensorFlow, one operation has multiple implementations that consume memory differently. TensorFlow selects an implementation depending on the available memory. For the right part, the simulator did a good job of simulating swap-in operations; in particular, the tail parts were quite similar. Finally, we can see that the number of allocation/deallocation steps in simulation was often smaller than that in real operation because an operation may allocate and deallocate its internal data.

5.3 Auto-tuning

Since the search space is huge, in this section, we propose a heuristic for exploring the search space. We search parameters in the order of *swapout_threshold*, *swapin_ahead*, and *swapin_groupby*. This is because, *swapout_threshold* is the most important for controlling the amount of data swapped out and in. It directly affects the peak memory consumption. In the context of deep learning, whose training is a memory-intensive task, we need to keep as much memory in the GPU as possible to increase locality and GPU utilization. Hence, the larger *swapout_threshold* is, the better training performance is. To search for a value for *swapout_threshold*, we use the lowest values for *swapin_ahead* and *swapin_groupby*, which are 1 and 0, respectively. After finding a suitable value for *swapout_threshold*, we use that value for *swapout_threshold*.

**Figure 6.** LMS module in TensorFlow**Listing 1.** Sample Python code for using LMS in TensorFlow

```

1 from lms import LMS
2 lms_obj = LMS(
3     swapout_threshold=18, swapin_ahead=5,
4     swapin_groupby=4)
5 lms_obj.run(graph=tf.get_default_graph())
6 with tf.Session() as sess:
7     sess.run(tf.global_variables_initializer())
8     batch = mnist.train.next_batch(50)
9     train_step.run(feed_dict={x: batch[0],
10                             y: batch[1]})

```

and still use the lowest value for *swapin_groupby*, that is, 0, and search for a value for *swapin_ahead* to increase the overlap between computation and data transfer. Finally, we use values that were found for *swapout_threshold* and *swapin_ahead*, and we search for a value for *swapin_groupby*.

For each parameter search, we found that binary search was effective enough. In each step of the binary search, we simulate the memory consumption for a specific set of parameter values. If the memory consumption becomes larger than the maximum memory for simulation, the current set of parameter values is not good, so we go to the lower range of values for the next search. Otherwise, we go to the upper range of values.

The maximum memory for simulation is the point beyond which we will get out-of-memory. Since we do not deal with learning parameters (variables), the available memory for simulation is calculated by subtracting the memory consumption of learning variables from the total physical memory. The ratio of the maximum memory for simulation to the available memory is controlled by a user-define value, say 0.9 or 0.8. This ratio is used for a simulator to safely simulate memory consumption under side-effect factors like the internal memory consumption of an operation, and it should vary depending on the neural model.

6 Integration into TensorFlow

We developed a TensorFlow module, called *Large Model Support (LMS)*, based on our proposed approach. The module allows users to quickly turn their large model into one that can be trained with limited GPU memory. In TensorFlow, users first define a neural network model. TensorFlow then automatically generates a computational graph from the model. Finally, users define a TensorFlow session to execute operations in the graph. Once a session is invoked, users cannot modify the graph. Hence, we implement LMS by inserting necessary operations into the graph automatically and statically before a session starts.

Table 3. Computational graphs for ResNet-50 and 3D U-Net

Model	Image size	Vertices	Edges	Topo. ordering level
ResNet-50	224 ²	3526	4685	344
3D U-Net	144 ³	14913	19595	1739
3D U-Net	256 ³	15480	20729	1739

Table 4. Maximum mini-batch size for ResNet-50 and maximum image size for 3D U-Net. ✗ means we got out-of-memory error. ✓ means we could train.

Model	Image size	Mini-batch size	No LMS	LMS
ResNet-50	224 ²	190	✓	✓
ResNet-50	224 ²	928	✗	✓
3D U-Net	144 ³	1	✓	✓
3D U-Net	256 ³	1	✗	✓

Figure 6 shows how LMS is positioned in TensorFlow, and Listing 1 shows a brief example of using LMS in TensorFlow. LMS takes a computational graph and automatically modifies it by using the transformation rules presented in Section 4 (Lines 1–5 in Listing 1). The modified graph is then executed by a TensorFlow session as normal (Lines 6–10 in Listing 1). LMS includes auto-tuning for automatically searching for graph-rewriting parameters. If parameters *swapout_threshold*, *swapin_ahead* and *swapin_groupby* are not specified when an LMS object is created, then the auto-tuning is executed to find values for these parameters.

7 Experiments

7.1 Experimental Environment

Experiments were run on an IBM POWER8 NUMA-based machine [10]. The machine had two 4-GHz 10-core POWER8 processors and 256 GB of RAM per processor. There were four NVIDIA Tesla P100 GPUs (each with 16 GB of memory), but we used only GPU 0 for our experiments. The GPU 0 was connected to a processor by two dedicated 80-GB/s duplex NVLinks. On the machine, we installed TensorFlow 1.10, CUDA Toolkit v9.0, and cuDNN 7.2.1.

We evaluated LMS by using two popular deep learning models: ResNet-50³ for image recognition and 3D U-Net⁴ [11] for image segmentation. We used a synthetic dataset for ResNet-50 and a brain tumor dataset for 3D U-Net [2, 3]. To make a model larger, we increased the mini-batch size of each iteration for ResNet-50, and we increased the image size for 3D U-Net. By default, we always fused swap-out operations.

³<https://github.com/tensorflow/benchmarks>

⁴<https://github.com/ellisdg/3DUnetCNN>

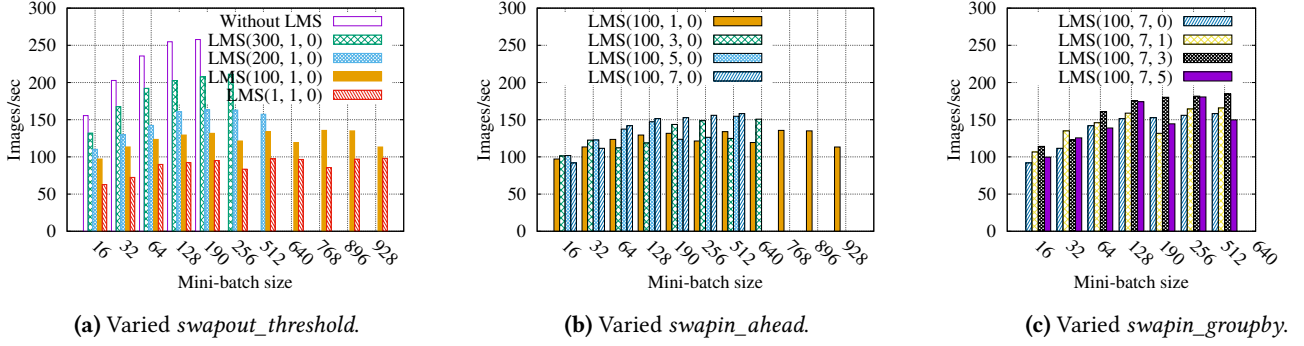


Figure 7. Effectiveness of parameters. $LMS(\text{swapout_threshold}, \text{swapin_ahead}, \text{swapin_groupby})$ denotes LMS run for specific set of graph-rewriting parameters. Points with no data mean they were failed with out-of-memory.

7.2 Analysis of Computational Graphs

Table 3 shows information on the computational graphs of ResNet-50 and 3D U-Net. ResNet-50 had 3526 operations in total, and its topological ordering had 344 levels. 3D U-Net was much larger than ResNet-50, and its architecture is changed in accordance with the input image size. For a 3D image size of 144^3 , 3D U-Net had 14913 operations in total with 1739 levels in the topological ordering. For a 3D image size of 256^3 , 3D U-Net had 15480 operations in total with 1739 levels in the topological ordering. Looking at the number of edges for both ResNet-50 and 3D U-Net, it is clear that operations were highly connected.

7.3 Maximum Mini-batch Size and Image Size

Table 4 shows the maximum mini-batch size we were able to train by using LMS. `swapout_threshold` was set to 1 to swap as many tensors out as possible. `swapin_ahead` was set to 1 to trigger swap-in operations as late as possible. `swapin_groupby` was set to 0 to turn off swap-in fusion. Since `swapout_threshold`=1, all edges with a distance starting from 2 were rewritten. For ResNet-50, the maximum mini-batch size we were able to train was 928, 4.9 times larger than the maximum mini-batch size the vanilla TensorFlow were able to train. For 3D U-Net, we were able to train with an image size of 256^3 instead of the 144^3 the vanilla TensorFlow was able to train, resulting in a 5.6 times larger image resolution. These results are better than those in previous works. In particular, the re-computing approach in [17] (implemented on top of MXNet [4]) achieved a 4x reduction in memory, and the swapping approach in [13] achieved only a 4.6x larger mini-batch size for ResNet-50. For 3D U-Net, we did not find any previous work on enabling its a large model, though this is an important application in the medical field.

7.4 Training Performance

Due to limited space in the paper and that ResNet-50 is a common benchmark in deep learning, we only analyze in detail results for ResNet-50. However, the effectiveness of

the rewriting parameters for 3D U-Net was relatively similar to the one for ResNet-50.

7.4.1 Effectiveness of `swapout_threshold`

Figure 7a shows the effectiveness of parameter `swapout_threshold` on the training performance of ResNet-50. We measured the number of images per second (images/sec) for each mini-batch size. We fixed parameters `swapin_ahead` and `swapin_groupby`, and we varied `swapout_threshold`. With `swapout_threshold`=1, we rewrote most of the edges in the graph and achieved the maximum mini-batch size of 928. Since many tensors were swapped out and in, we saw a big degradation in performance compared with the highest performance at the mini-batch size 190 (without LMS). To reduce the number of tensors swapped, we increased the threshold to 100, 200, and 300. It was clear that the performance was improved when the threshold was increased. In particular, with `swapout_threshold`=300, we had only a 19% performance degradation at a mini-batch size of 190. We even achieved the same performance at a mini-batch size of 256. These results were also better than the re-computing technique, which had an additional 30% runtime due to the double forward cost in gradient calculation [17].

7.4.2 Effectiveness of `swapin_ahead`

Figure 7b shows the results when we varied `swapin_ahead`. We fixed `swapout_threshold`=100 to have more memory space for changing `swapin_ahead`. Recall that the larger `swapin_ahead` is, the higher memory consumption is. When we increased `swapin_ahead`, the maximum mini-batch size we were able to train was decreased. In particular, the maximum mini-batch size was just 640 when `swapin_ahead`=3 and was decreased to 512 when `swapin_ahead`=5. However, a large value for `swapin_ahead` means we have more of an overlap of data transfer and computation. This results in better performance with a larger `swapin_ahead`. In this paper, we fixed the same `swapin_ahead` for every swap-in operation.

Table 5. Results with auto-tuning. “Best in Fig. 7” means best performance for mini-batch size in Fig. 7. Values in parentheses denote (*swapout_threshold*, *swapin_ahead*, *swapin_groupby*). “OFF”: Auto-tuning turned LMS off. “NF”: Auto-tuning did not find values. “OOM”: got out-of-memory. “Auto-tuning-{0.8, 0.95}”: Memory ratio for simulation was 0.8 or 0.95.

Mini-batch size	16	32	64	128	190	256	512	640	768	896	928
No LMS (images/sec)	156	203	235	255	258	OOM	OOM	OOM	OOM	OOM	OOM
Best in Fig. 7 (images/sec)						211	185	151	136	135	113
Graph-rewriting params						(300,1,0)	(100,7,3)	(100,3,0)	(100,1,0)	(100,1,0)	(100,1,0)
Auto-tuning-0.8 (images/sec)	156	203	235	255	241	236	209	167	172	OOM	OOM
Graph-rewriting params	OFF	OFF	OFF	OFF	(314,62,344)	(280,22,344)	(202,7,344)	(180,2,344)	(160,1,344)	NF	NF
Auto-tuning time (sec)					13.3	14.5	17.9	18.5	20.3	19.2	22.6
The number of simulations					23	23	23	23	24	23	23
Minimum simulation time (sec)					0.3	0.3	0.3	0.3	0.4	0.4	0.4
Maximum Simulation time (sec)					1.0	1.5	3.2	3.5	3.5	3.5	3.5
Auto-tuning-0.95 (images/sec)										138	137
Graph-rewriting params										(150,1,0)	(150,1,0)

To achieve the best performance, it would be ideal to search for the best *swapin_ahead* for *each swap-in operation*, which requires runtime information on tensors and operations.

7.4.3 Effectiveness of *swapin_groupby*

Figure 7c shows the results obtained when we varied *swapin_groupby*. Parameter *swapin_groupby* helps us with close swap-in operations, which slows down performance. By increasing *swapin_groupby*, more swap-in tensors are reused, resulting in better performance. Looking at Figure 7c, it seems that we achieved the best performance with *swapin_groupby*=3. With *swapin_groupby*=5, performance was not good. This may be due to algorithm selection in the cuDNN library. In particular, when we used a large *swapin_groupby*, more tensors were kept in the GPU for a long time for re-use, which reduced the available memory in the GPU and caused cuDNN to choose a slower algorithm for each operation.

Overall, considering *swapin_threshold*=100 for a mini-batch size of 512 in Fig. 7a, we were able to quickly get a better performance by tuning *swapin_ahead* and *swapin_groupby*. In particular, the number of images per second for LMS(100, 1, 0) was increased from 134 (Fig. 7a) to 185 (Fig. 7c) by using LMS(100, 7, 3), resulting in a 38% improvement in performance.

7.4.4 Effectiveness of Auto-tuning

We evaluated the effectiveness of the simulation-based auto-tuning by comparing training times for parameters found by the auto-tuning and the best training times we got in the previous experiments in Fig. 7. Table 5 summarizes our results.

For mini-batch sizes from 16 to 190 that fit in the GPU memory, the auto-tuning correctly recognized that LMS was not necessary for these mini-batch sizes, except for 190. At the mini-batch size of 190, TensorFlow recognized that it

was running out-of-memory, and it tried its own heuristic to allocate memory. In that sense, the auto-tuning was correct when it reported an out-of-memory error for 190. Graph-rewriting parameters that were found by the auto-tuning for 190 said that the training was running out-of-memory just a bit because *swapout_threshold* was very large and close to the number of topological ordering levels, and *swapin_ahead* was very large too. In this case, the training time with auto-tuning was slightly slower than that without LMS.

For mini-batch sizes from 256 to 768 that fit in the GPU memory, the auto-tuning found graph-rewriting parameters with which the training times were better than the best training times in Fig. 7. We can see that the auto-tuning tried to decrease the value of *swapout_threshold* when the mini-batch size increased. Also, the chance for overlapping communication and computation was reduced since the value of *swapin_ahead* decreased. The auto-tuning always found a value of 344 (that is, the number of topological ordering levels) for *swapin_groupby*, which means we had enough memory for fusing swap-in operations in order to maximize the performance. Let us consider the mini-batch size of 512 with which we did some manual tuning in Fig. 7. We achieved a 38% improvement in performance (from 134 to 185 images/sec). However, the auto-tuning could find graph-rewriting parameters that resulted in an additional improvement of 13% (from 185 to 209 images/sec).

For very large mini-batch sizes, 896 and 928, the auto-tuning failed to find graph-rewriting parameters. Even with the successful results in Fig. 7, TensorFlow always printed out messages regarding out-of-memory and tried its heuristic for these mini-batch sizes. Hence, we increased the memory ratio for simulation from 0.8 to 0.95 to see if the auto-tuning could find graph-rewriting parameters for these large batch sizes. As shown in Table 5, the auto-tuning with a memory ratio of 0.95 found that the values for the graph-rewriting

parameters and the training times were better than the best training times in Fig. 7.

Table 5 also shows the cost of auto-tuning. We measured the auto-tuning time for each mini-batch size. One auto-tuning run invoked multiple simulations. The number of simulations was the parameter space the auto-tuning was exploring. Simulation time depends on when the simulation detects OOM. Hence, we measured the minimum and maximum times of simulation. We observed that the cost of auto-tuning was very small, 13 to 23 seconds, compared with the whole training that may take days to weeks.

8 Conclusion

We proposed a formal approach to deriving swap-out and swap-in operations that enables support for large models. We formally defined the concept of the computational graph and used the concepts from the theory of program transformation to derive new operations as well as optimize the graph. The derivation was for reducing the distance between two operations in a categorized topological ordering, which makes it possible for a large model to fit GPU memory. We proposed a simulation-based auto-tuning for automatically finding graph-rewriting parameters. The auto-tuning worked well with a practical neural network, ResNet-50. The experimental results showed that our approach helped train very large models, i.e., a 4.9 times larger mini-batch size for ResNet-50 and a 5.6 times larger image resolution for 3D U-Net.

Though our definition of the computational graph is inspired by TensorFlow, it is still general enough to be integrated into other graph-based frameworks. In the future, we will do such integration. As we mentioned, we aim to provide a strong foundation for large model support. To conserve paper space, we do not go into further integrating other techniques such as neural network compression and re-computing, though we quickly mentioned compression for CPU memory in Sec. 4.3. Furthermore, we believe our approach would work well with data parallelism without any modification if users would like to do training on multiple GPUs. In such a case, graph rewriting can be applied to each computational graph on each GPU independently. Investigating novel techniques for finding control dependency operations for *each operation* is an open, interesting problem.

References

- [1] Martín Abadi, Michael Isard, and Derek G. Murray. 2017. A Computational Model for TensorFlow: An Introduction. In *the 1st ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*. 1–7.
- [2] Spyridon Bakas, Hamed Akbari, Aristeidis Sotiras, Michel Bilello, Martin Rozycki, Justin Kirby, John Freymann, Keyvan Farahani, and Christos Davatzikos. 2017. Segmentation Labels and Radiomic Features for the Pre-operative Scans of the TCGA-GBM collection. *The Cancer Imaging Archive* (2017). <https://doi.org/10.7937/K9/TCIA.2017.KLXWJJ1Q>
- [3] Spyridon Bakas, Hamed Akbari, Aristeidis Sotiras, Michel Bilello, Martin Rozycki, Justin Kirby, John Freymann, Keyvan Farahani, and Christos Davatzikos. 2017. Segmentation Labels and Radiomic Features for the Pre-operative Scans of the TCGA-LGG collection. *The Cancer Imaging Archive* (2017). <https://doi.org/10.7937/K9/TCIA.2017.GJQ7R0EF>
- [4] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. 2015. MXNet: A Flexible and Efficient Machine Learning Library for Heterogeneous Distributed Systems. *CoRR* abs/1512.01274 (2015). <http://arxiv.org/abs/1512.01274>
- [5] Yoojin Choi, Mostafa El-Khamy, and Jungwon Lee. 2018. Universal Deep Neural Network Compression. *CoRR* abs/1802.02271 (2018). <http://arxiv.org/abs/1802.02271>
- [6] Özgün Çiçek, Ahmed Abdulkadir, Soeren S. Lienkamp, Thomas Brox, and Olaf Ronneberger. 2016. 3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation. *CoRR* abs/1606.06650 (2016). <http://arxiv.org/abs/1606.06650>
- [7] Julian Faraone, Nicholas J. Fraser, Giulio Gamberdella, Michaela Blott, and Philip Heng Wai Leong. 2017. Compressing Low Precision Deep Neural Networks Using Sparsity-Induced Regularization in Ternary Networks. *CoRR* abs/1709.06262 (2017). <http://arxiv.org/abs/1709.06262>
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. *CoRR* abs/1512.03385 (2015). <http://arxiv.org/abs/1512.03385>
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Identity Mappings in Deep Residual Networks. In *European Conference on Computer Vision (ECCV)*. 630–645.
- [10] IBM. 2016. IBM Power System S822LC for High Performance Computing. <http://www-03.ibm.com/systems/power/hardware/s822lc-hpc/>.
- [11] Fabian Isensee, Philipp Kickingereder, Wolfgang Wick, Martin Bendzsus, and Klaus H. Maier-Hein. 2018. Brain Tumor Segmentation and Radiomics Survival Prediction: Contribution to the BRATS 2017 Challenge. *CoRR* abs/1802.10508 (2018). <http://arxiv.org/abs/1802.10508>
- [12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *International Conference on Neural Information Processing Systems (NIPS)*. 1097–1105.
- [13] Chen Meng, Minmin Sun, Jun Yang, Minghui Qiu, and Yang Gu. 2017. Training Deeper Models by GPU Memory Optimization on TensorFlow. In *ML Systems Workshop in NIPS*.
- [14] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. 2016. vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design. *CoRR* abs/1602.08124 (2016). <http://arxiv.org/abs/1602.08124>
- [15] Nikolay Sakharnykh. 2017. Unified Memory on Pascal and Volta. (2017). GTC.
- [16] Koichi Shirahata, Yasumoto Tomita, and Atsushi Ike. 2016. Memory Reduction Method for Deep Neural Network Training. In *the 26th International Workshop on Machine Learning for Signal Processing (MLSP)*. 1–6.
- [17] Chiyuan Zhang Carlos Guestrin Tianqi Chen, Bing Xu. 2016. Training Deep Nets with Sublinear Memory Cost. *ArXiv e-prints* (April 2016). [arXiv:1604.06174](https://arxiv.org/abs/1604.06174)
- [18] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU Memory Management for Training Deep Neural Networks. In *the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. 41–53.